

ELEC 313: Digital Systems Design II

Project

Saeed Ba Yaaqoob

ID: 40264986

Instructor: Ted Obuchowicz

Section: AA

Date Due: June 29, 2026

Table of Contents

Objective.....	3
Procedures.....	3
Result and Discussion	4-6
Conclusion.....	6
Appendix.....	.7
Appendix A: Part 1 VHDL Code	7-12
Appendix B: Modelsim simulation for PART 1	13
Appendix C: Capture of Is -al bit file proof PART 1.....	14
Appendix D: The ASM chart and complete block diagram of control unit.....	15-16
Appendix E: Part 2 VHDL Code	17-23
Appendix F: Modelsim simulation for PART 2.....	24-25
Appendix G: Capture of Is -al bit file proof PART 2.....	26

I will also attach an additional printed pdf modelsimum simulation at back for both parts

Project Report

Objective

The objective of this project is to design the $3k+1$ algorithm using VHDL. In Part 1, the algorithm is designed, simulated, and synthesized to determine the smallest positive integer whose sequence contains at least nine terms. In Part 2, the complete digital system is designed using the ASM chart design methodology. The ASM chart is then implemented in VHDL and tested on the board to verify the correct operation of the control unit, datapath, and seven-segment display.

Procedures

1- Part 1

The VHDL design was written and compiled in ModelSim. The circuit was simulated using a do file to verify that the design operated correctly before synthesis. After successful simulation, the VHDL source file together with the XDC constraints file were added to a Vivado project. The design was synthesized and implemented to generate the bitstream file for the board.

2- Part 2

The ASM chart was first developed to describe the operation of the digital system. Based on the ASM chart, the datapath and control unit were implemented in VHDL and integrated into one design. The completed design was then simulated, synthesized, and downloaded to the board to verify the operation of the seven-segment display and the overall system.

Result and Discussion

1- Part 1 (Appendix A, B, and C)

The VHDL code shown in Appendix A implements the $3k+1$ algorithm using three clocked processes and an asynchronous reset. The first process divides the 100 MHz onboard clock to generate a 1 Hz enable signal so the algorithm advances once every second on the FPGA board. The second process implements the $3k+1$ algorithm by storing the current number, current term, sequence length, and done signal in registers. The algorithm applies the required operations for odd and even numbers and continues until the sequence reaches 1. If the sequence contains fewer than nine terms, the next positive integer is tested. Otherwise, the done signal becomes high and the registers stop updating. The third process controls the multiplexing of the seven-segment display so the current number and term are displayed correctly on the FPGA board.

In Appendix B, we can see the simulation waveform. At the beginning, the reset signal is activated, therefore the initial values are `number_reg = 1`, `term_reg = 1`, `length_reg = 1`, and `done_reg = 0`. After the reset is released, the design starts testing each positive integer and generates its corresponding $3k+1$ sequence. The waveform shows that `number_reg` increases from 1 until 6, while `term_reg` follows the correct $3k+1$ sequence for each number. When number 6 is tested, the sequence reaches 9 terms, therefore `done_reg` changes to logic 1 and all registers stop updating. During the simulation, one problem was noticed when printing the waveform because some values disappeared when using the full range option. This problem was solved by selecting Current View, which displayed all waveform values correctly. Therefore, the simulation confirms that the design operates correctly.

The synthesis was completed successfully. Appendix C shows the output of the command `ls -al *.bit`, which confirms that the generated bitstream file exists in the project directory including date and time.

2- Part 2 (Appendix D, E, F, and G)

The completed ASM chart and the datapath and control unit block diagram are shown in **Appendix D**. The ASM chart describes the sequence of states used to implement the $3k+1$ algorithm, while the completed datapath shows the hardware blocks, control signals, and status signals used by the control unit. Together, they provide the design methodology used before writing the VHDL code and show how the control unit communicates with the datapath.

The VHDL implementation is presented in **Appendix E**. Unlike Part 1, the design is divided into a control unit and a datapath using the ASM methodology. The control unit is implemented as a finite-state machine that generates the control signals `reset_number`, `inc_number`, `load_term`, `shift_term`, `mult_add_term`, `reset_length`, `inc_length`, and `load_done`. These signals control the datapath registers, while the status signals `term_eq_1`, `term_even`, and `length_ge_9` are returned to the control unit to determine the next state. A separate clock divider generates the 1 Hz update signal, and another process multiplexes the seven-segment display to continuously show the current values of number and term.

The ModelSim simulation results are shown in **Appendix F**. After reset is asserted, the registers are initialized to `number = 1`, `term = 1`, `length = 1`, and `done = 0`. The waveform shows the control unit progressing through the ASM states while generating the required control signals during each clock cycle. The datapath registers update correctly according to these control signals, producing the expected $3k+1$ sequence. The simulation continues until `number = 6` reaches a sequence length of 9, where `done_reg` and `done_out` become high, confirming that both the control unit and datapath operate correctly according to the ASM design.

The synthesis proof for Part 2 is provided in **Appendix G**. The Linux terminal command `ls -al *.bit` was used to verify that the generated .bit file exists together with its timestamp, confirming that the FPGA bitstream was successfully created.

Conclusion

In this project, the main objective was successfully completed by designing and implementing the $3k+1$ sequence generator in VHDL. In Part 1, the algorithm was implemented using RTL VHDL and tested through simulation before preparing it for FPGA implementation. In Part 2, the same design was implemented again using the ASM chart methodology by separating the datapath and the control unit. The simulation results and synthesis confirmed that both designs worked correctly and produced the expected output. From this project, I learned how to design a digital system using two different methods and understand the difference between a direct RTL design and an ASM based design. I also learned how the control unit and datapath work together using control and status signals, how to simulate and debug the design in ModelSim, and how to prepare the design for synthesis on the board. Overall, this project helped me improve my VHDL programming and gave me more experience with digital system design.

Appendix A: Part 1 VHDL Code

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity three_k_plus_one is
    port(
        reset      : in  std_logic;
        clk_in     : in  std_logic;
        an         : out std_logic_vector(7 downto 0);
        sseg       : out std_logic_vector(7 downto 0);
        done_out   : out std_logic
    );
end three_k_plus_one;

architecture rtl of three_k_plus_one is

    signal number_reg : unsigned(6 downto 0);
    signal term_reg   : unsigned(6 downto 0);
    signal length_reg : unsigned(3 downto 0);
    signal done_reg   : std_logic;

    signal count_1hz : unsigned(26 downto 0);
    signal tick_1hz  : std_logic;

    signal count_scan : unsigned(13 downto 0);
    signal tick_scan  : std_logic;
    signal scan_digit : unsigned(2 downto 0);
```

```

    signal hex_digit    : std_logic_vector(3 downto 0);

    function hex_to_sseg(x : std_logic_vector(3 downto 0)) return
std_logic_vector is
    begin
        case x is
            when "0000" => return "00000011";
            when "0001" => return "10011111";
            when "0010" => return "00100101";
            when "0011" => return "00001101";
            when "0100" => return "10011001";
            when "0101" => return "01001001";
            when "0110" => return "01000001";
            when "0111" => return "00011111";
            when "1000" => return "00000001";
            when "1001" => return "00001001";
            when "1010" => return "00010001";
            when "1011" => return "11000001";
            when "1100" => return "01100011";
            when "1101" => return "10000101";
            when "1110" => return "01100001";
            when others => return "01110001";
        end case;
    end function;

begin

    done_out <= done_reg;

    process(clk_in, reset)
    begin

```



```

    if reset = '1' then
        count_1hz <= (others => '0');
        tick_1hz <= '0';
    elsif clk_in'event and clk_in = '1' then
        if count_1hz = to_unsigned(99999999, count_1hz'length) then
            count_1hz <= (others => '0');
            tick_1hz <= '1';
        else
            count_1hz <= count_1hz + 1;
            tick_1hz <= '0';
        end if;
    end if;
end process;

process(clk_in, reset)
    variable next_number : unsigned(6 downto 0);
begin
    if reset = '1' then
        number_reg <= to_unsigned(1, 7);
        term_reg    <= to_unsigned(1, 7);
        length_reg  <= to_unsigned(1, 4);
        done_reg    <= '0';
    elsif clk_in'event and clk_in = '1' then
        if tick_1hz = '1' then
            if done_reg = '0' then
                if term_reg = to_unsigned(1, 7) then
                    if length_reg >= to_unsigned(9, 4) then
                        done_reg <= '1';
                    else
                        next_number := number_reg + 1;
                    end if;
                end if;
            end if;
        end if;
    end if;
end process;

```

```

        number_reg <= next_number;
        term_reg <= next_number;
        length_reg <= to_unsigned(1, 4);
    end if;
else
    if term_reg(0) = '0' then
        term_reg <= shift_right(term_reg, 1);
    else
        term_reg <= resize((term_reg *
to_unsigned(3, 7)) + to_unsigned(1, 14), 7);
    end if;
    length_reg <= length_reg + 1;
end if;
end if;
end if;
end if;
end process;

process(clk_in, reset)
begin
    if reset = '1' then
        count_scan <= (others => '0');
        tick_scan <= '0';
        scan_digit <= (others => '0');
    elsif clk_in'event and clk_in = '1' then
        if count_scan = to_unsigned(12499, count_scan'length) then
            count_scan <= (others => '0');
            tick_scan <= '1';
            scan_digit <= scan_digit + 1;
        else
            count_scan <= count_scan + 1;

```

```

        tick_scan <= '0';
    end if;
end if;
end process;

process(scan_digit, term_reg, number_reg)
begin
    an <= "11111111";
    hex_digit <= "0000";

    case scan_digit is
        when "000" =>
            an <= "11111110";
            hex_digit <= std_logic_vector(term_reg(3 downto 0));

        when "001" =>
            an <= "11111101";
            hex_digit <= '0' & std_logic_vector(term_reg(6 downto
4));

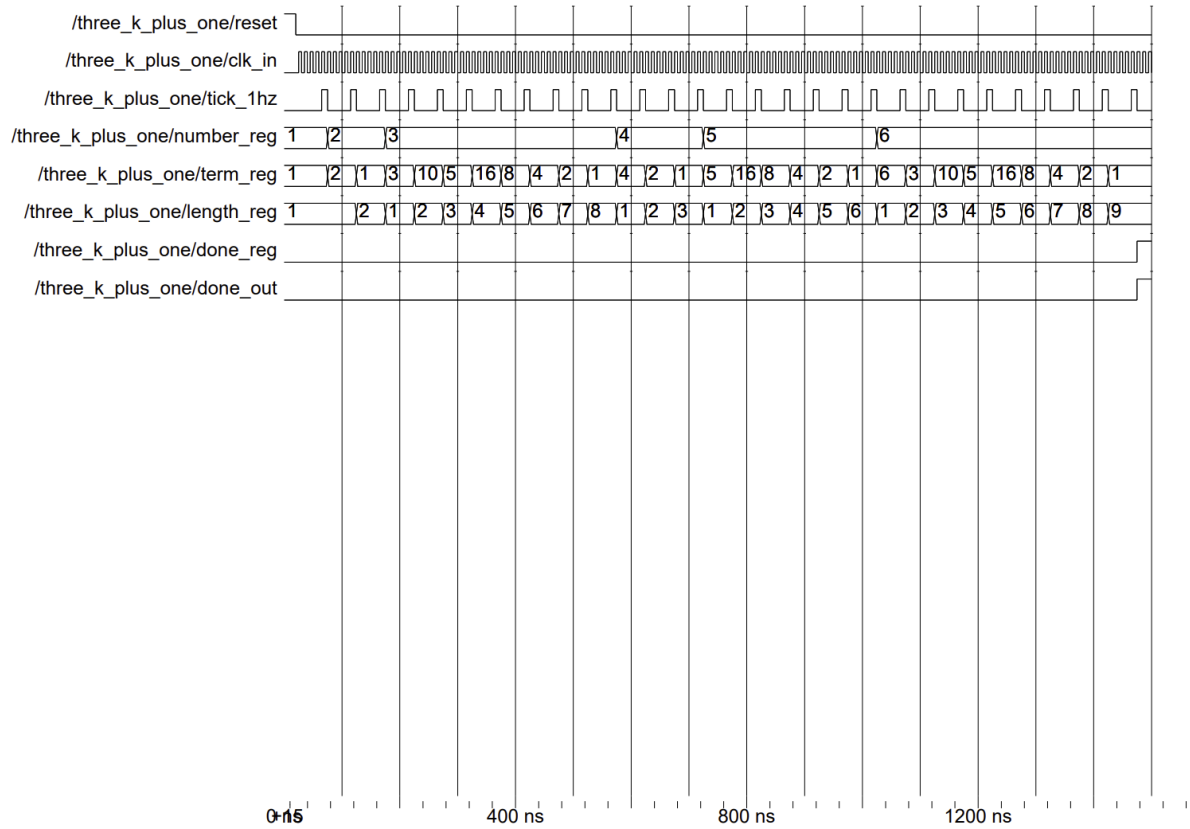
        when "010" =>
            an <= "11111011";
            hex_digit <= '0' & std_logic_vector(number_reg(2 downto
0));

        when others =>
            an <= "11111111";
            hex_digit <= "0000";
    end case;
end process;

```

```
sseg <= hex_to_sseg(hex_digit);  
  
end rtl;
```

Appendix B: Modelsim simulation for PART 1 (Also printed at back)



Entity:three_k_plus_one Architecture:rtl Date: Sun Jun 28 06:28:27 AM EDT 2026 Row: 1 Page: 1

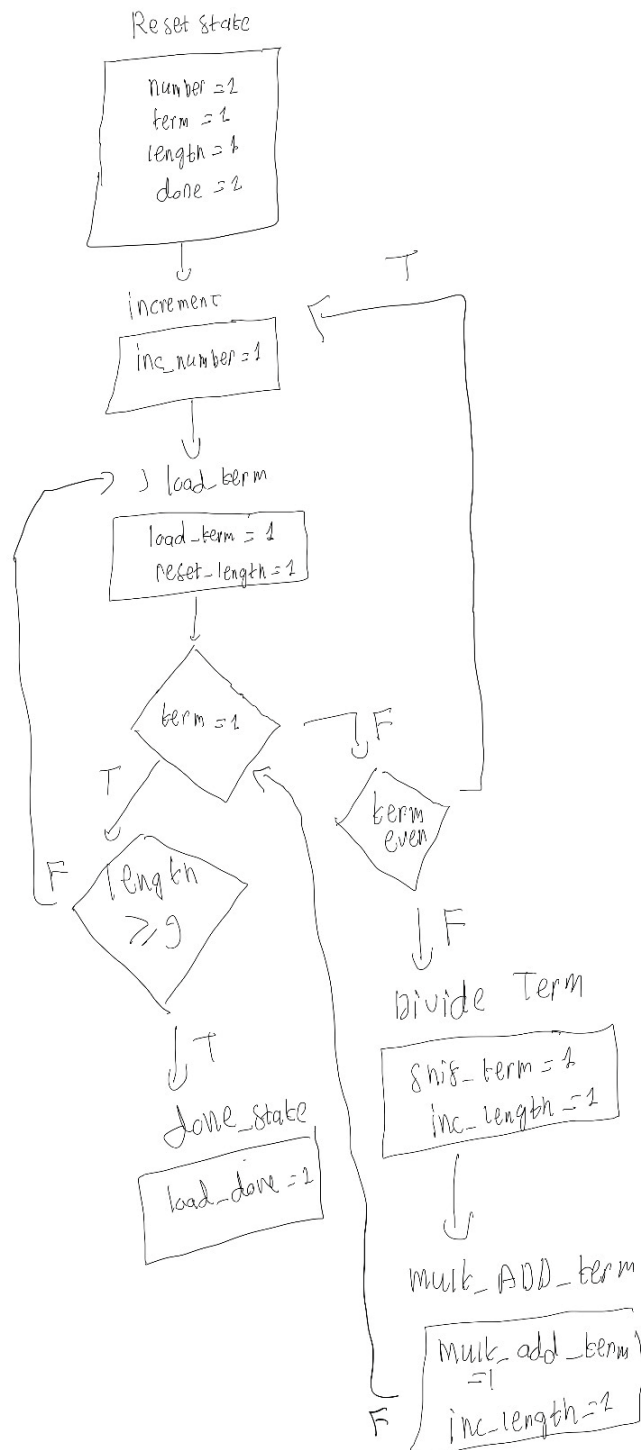
Appendix C: Capture of ls -al bit file proof PART 1

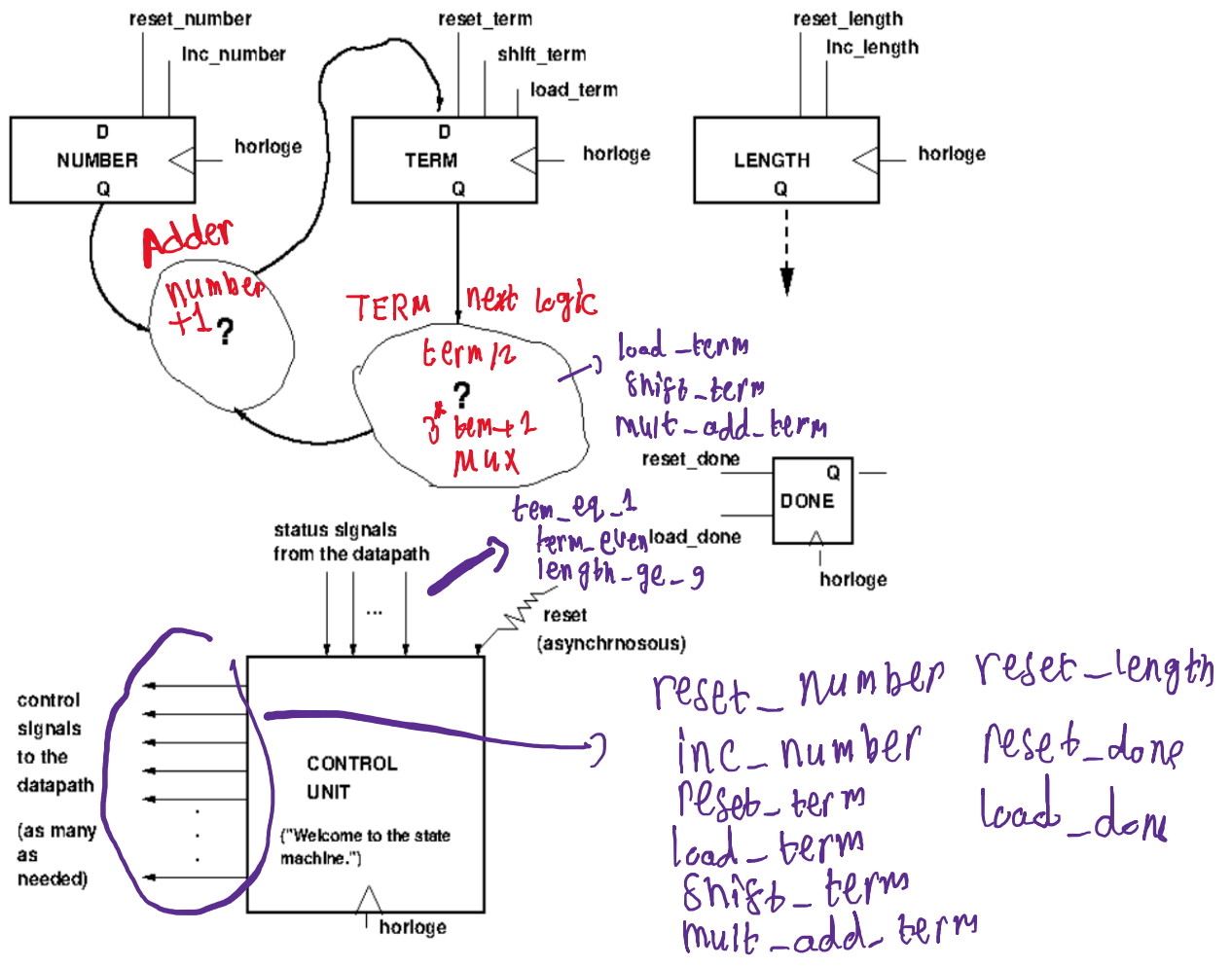
```
[poise] [/home/s/s_bayaaq/COEN313/project/project_part1/project_part1.runs/impl_1] > pwd
/nfs/home/s/s_bayaaq/COEN313/project/project_part1/project_part1.runs/impl_1
[poise] [/home/s/s_bayaaq/COEN313/project/project_part1/project_part1.runs/impl_1] > ls -al *.bit
-rw-rw---- 1 s_bayaaq s_bayaaq 3825917 Jun 28 15:31 three_k_plus_one.bit
[poise] [/home/s/s_bayaaq/COEN313/project/project_part1/project_part1.runs/impl_1] >
```

Then I rename it to process.bit as required:

```
[poise] [/home/s/s_bayaaq/COEN313/project/project_part1/project_part1.runs/impl_1] > ls -al *.bit
-rw-rw---- 1 s_bayaaq s_bayaaq 3825917 Jun 28 15:59 process.bit
-rw-rw---- 1 s_bayaaq s_bayaaq 3825917 Jun 28 15:31 three_k_plus_one.bit
[poise] [/home/s/s_bayaaq/COEN313/project/project_part1/project_part1.runs/impl_1] > |
```

Appendix D: The ASM chart and complete block diagram of control unit





Appendix E: Part 2 VHDL Code

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity three_k_plus_one is
  port(
    reset      : in  std_logic;
    clk_in     : in  std_logic;
    an         : out std_logic_vector(7 downto 0);
    sseg       : out std_logic_vector(7 downto 0);
    done_out   : out std_logic
  );
end three_k_plus_one;

architecture asm_rtl of three_k_plus_one is

  constant ONE_HZ_MAX : natural := 4;
  constant SCAN_MAX   : natural := 12499;

  type state_type is (
    reset_state,
    increment_number,
    load_term_state,
    test_term_state,
    check_length_state,
    test_even_state,
    divide_term_state,
    mult_add_state,
    done_state
  );

  signal state_reg, state_next : state_type;

  signal number_reg : unsigned(6 downto 0);
  signal term_reg   : unsigned(6 downto 0);
  signal length_reg : unsigned(3 downto 0);
  signal done_reg   : std_logic;

  signal count_1hz : unsigned(26 downto 0);
  signal tick_1hz  : std_logic;
```

```

signal count_scan : unsigned(13 downto 0);
signal scan_digit : unsigned(2 downto 0);

signal reset_number      : std_logic;
signal inc_number        : std_logic;
signal reset_term        : std_logic;
signal load_term         : std_logic;
signal shift_term        : std_logic;
signal mult_add_term     : std_logic;
signal reset_length      : std_logic;
signal inc_length        : std_logic;
signal reset_done        : std_logic;
signal load_done         : std_logic;

signal term_eq_1      : std_logic;
signal term_even      : std_logic;
signal length_ge_9    : std_logic;

signal hex_digit : std_logic_vector(3 downto 0);

begin

    done_out <= done_reg;

    term_eq_1  <= '1' when term_reg = to_unsigned(1, 7) else
'0';
    term_even  <= '1' when term_reg(0) = '0' else '0';
    length_ge_9 <= '1' when length_reg >= to_unsigned(9, 4) else
'0';

    process(clk_in, reset)
    begin
        if reset = '1' then
            count_1hz <= (others => '0');
            tick_1hz <= '0';
        elsif clk_in'event and clk_in = '1' then
            if count_1hz = to_unsigned(ONE_HZ_MAX,
count_1hz'length) then
                count_1hz <= (others => '0');
                tick_1hz <= '1';
            else
                count_1hz <= count_1hz + 1;
                tick_1hz <= '0';
            end if;
        end if;
    end process;

```

```

        end if;
    end process;

    process(clk_in, reset)
    begin
        if reset = '1' then
            state_reg <= reset_state;
        elsif clk_in'event and clk_in = '1' then
            if tick_1hz = '1' then
                state_reg <= state_next;
            end if;
        end if;
    end process;

    process(state_reg, term_eq_1, term_even, length_ge_9)
    begin
        state_next <= state_reg;

        reset_number    <= '0';
        inc_number       <= '0';
        reset_term       <= '0';
        load_term        <= '0';
        shift_term       <= '0';
        mult_add_term    <= '0';
        reset_length     <= '0';
        inc_length       <= '0';
        reset_done       <= '0';
        load_done        <= '0';

        case state_reg is

            when reset_state =>
                reset_number <= '1';
                reset_term   <= '1';
                reset_length <= '1';
                reset_done   <= '1';
                state_next <= increment_number;

            when increment_number =>
                inc_number <= '1';
                state_next <= load_term_state;

            when load_term_state =>
                load_term <= '1';

```

```

        reset_length <= '1';
        state_next <= test_term_state;

    when test_term_state =>
        if term_eq_1 = '1' then
            state_next <= check_length_state;
        else
            state_next <= test_even_state;
        end if;

    when check_length_state =>
        if length_ge_9 = '1' then
            state_next <= done_state;
        else
            state_next <= increment_number;
        end if;

    when test_even_state =>
        if term_even = '1' then
            state_next <= divide_term_state;
        else
            state_next <= mult_add_state;
        end if;

    when divide_term_state =>
        shift_term <= '1';
        inc_length <= '1';
        state_next <= test_term_state;

    when mult_add_state =>
        mult_add_term <= '1';
        inc_length <= '1';
        state_next <= test_term_state;

    when done_state =>
        load_done <= '1';
        state_next <= done_state;

    end case;
end process;

process(clk_in, reset)
begin
    if reset = '1' then

```

```

        number_reg <= to_unsigned(1, 7);
    elsif clk_in'event and clk_in = '1' then
        if tick_1hz = '1' then
            if reset_number = '1' then
                number_reg <= to_unsigned(1, 7);
            elsif inc_number = '1' then
                number_reg <= number_reg + 1;
            end if;
        end if;
    end if;
end process;

process(clk_in, reset)
begin
    if reset = '1' then
        term_reg <= to_unsigned(1, 7);
    elsif clk_in'event and clk_in = '1' then
        if tick_1hz = '1' then
            if reset_term = '1' then
                term_reg <= to_unsigned(1, 7);
            elsif load_term = '1' then
                term_reg <= number_reg;
            elsif shift_term = '1' then
                term_reg <= shift_right(term_reg, 1);
            elsif mult_add_term = '1' then
                term_reg <= resize((term_reg * to_unsigned(3,
7)) + to_unsigned(1, 14), 7);
            end if;
        end if;
    end if;
end process;

process(clk_in, reset)
begin
    if reset = '1' then
        length_reg <= to_unsigned(1, 4);
    elsif clk_in'event and clk_in = '1' then
        if tick_1hz = '1' then
            if reset_length = '1' then
                length_reg <= to_unsigned(1, 4);
            elsif inc_length = '1' then
                length_reg <= length_reg + 1;
            end if;
        end if;
    end if;
end process;

```

```

        end if;
    end process;

    process(clk_in, reset)
    begin
        if reset = '1' then
            done_reg <= '0';
        elsif clk_in'event and clk_in = '1' then
            if tick_1hz = '1' then
                if reset_done = '1' then
                    done_reg <= '0';
                elsif load_done = '1' then
                    done_reg <= '1';
                end if;
            end if;
        end if;
    end process;

    process(clk_in, reset)
    begin
        if reset = '1' then
            count_scan <= (others => '0');
            scan_digit <= (others => '0');
        elsif clk_in'event and clk_in = '1' then
            if count_scan = to_unsigned(SCAN_MAX,
count_scan'length) then
                count_scan <= (others => '0');
                scan_digit <= scan_digit + 1;
            else
                count_scan <= count_scan + 1;
            end if;
        end if;
    end process;

    process(scan_digit, term_reg, number_reg)
    begin
        an <= "11111111";
        hex_digit <= "0000";

        case scan_digit is
            when "000" =>
                an <= "11111110";
                hex_digit <= std_logic_vector(term_reg(3 downto
0));

```

```

        when "001" =>
            an <= "11111101";
            hex_digit <= '0' & std_logic_vector(term_reg(6
downto 4));

        when "010" =>
            an <= "11111011";
            hex_digit <= std_logic_vector(number_reg(3 downto
0));

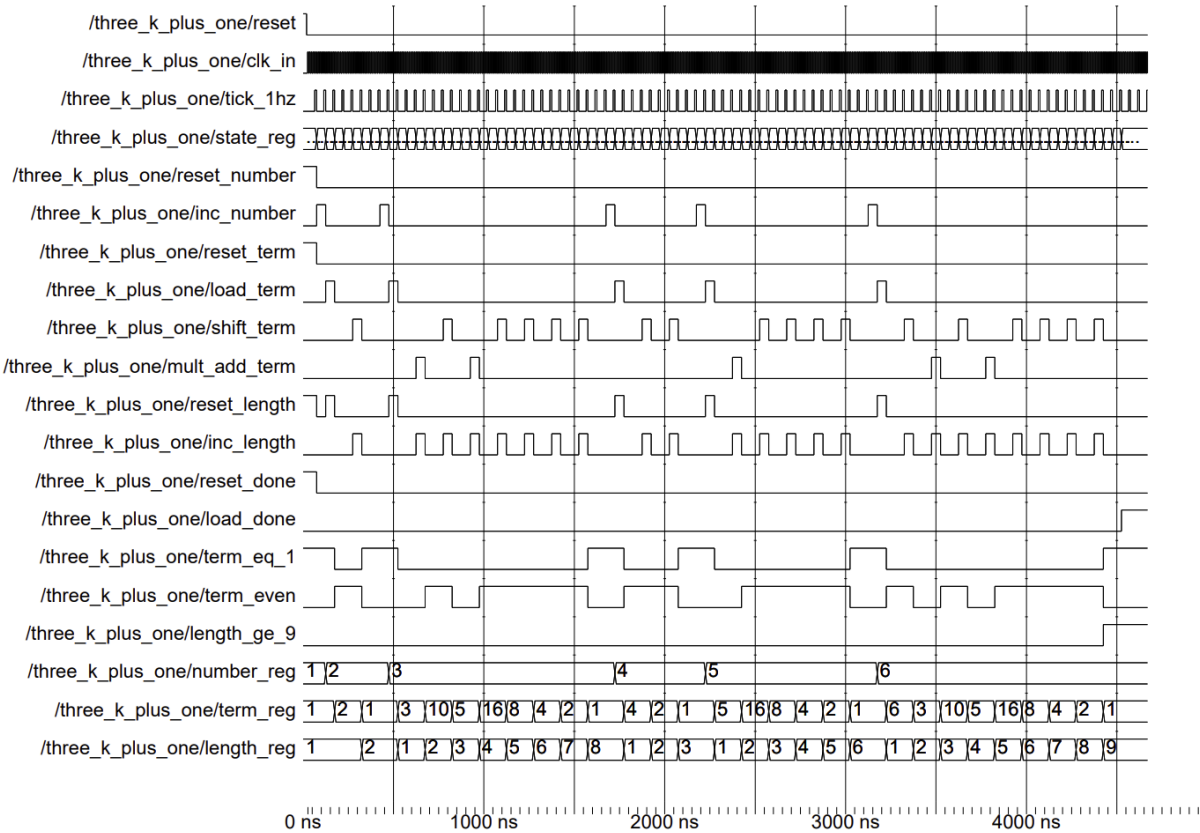
        when others =>
            an <= "11111111";
            hex_digit <= "0000";
        end case;
    end process;

    process(hex_digit)
    begin
        case hex_digit is
            when "0000" => sseg <= "00000011";
            when "0001" => sseg <= "10011111";
            when "0010" => sseg <= "00100101";
            when "0011" => sseg <= "00001101";
            when "0100" => sseg <= "10011001";
            when "0101" => sseg <= "01001001";
            when "0110" => sseg <= "01000001";
            when "0111" => sseg <= "00011111";
            when "1000" => sseg <= "00000001";
            when "1001" => sseg <= "00001001";
            when "1010" => sseg <= "00010001";
            when "1011" => sseg <= "11000001";
            when "1100" => sseg <= "01100011";
            when "1101" => sseg <= "10000101";
            when "1110" => sseg <= "01100001";
            when others => sseg <= "01110001";
        end case;
    end process;

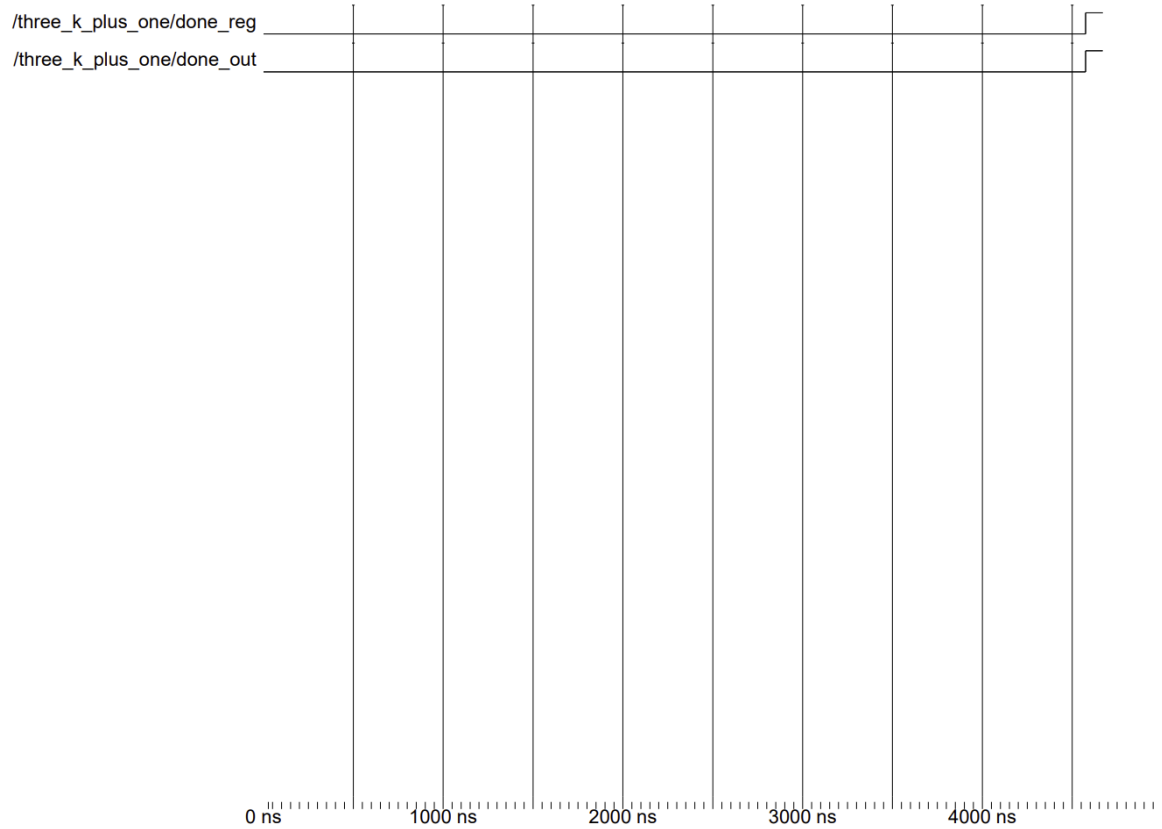
end asm_rtl;

```

Appendix F: Modelsim simulation for PART 2 (Also printed at back)



Entity:three_k_plus_one Architecture:asm_rtl Date: Sun Jun 28 01:12:30 PM EDT 2026 Row: 1 Page: 1



Entity:three_k_plus_one Architecture:asm_rtl Date: Sun Jun 28 01:12:30 PM EDT 2026 Row: 1 Page: 2

Appendix G: Capture of ls -al bit file proof PART 2

```
[poise] [/home/s/s_bayaaq/COEN313/project_ii/project_part2/project_part2.runs/impl_1] > pwd
/nfs/home/s/s_bayaaq/COEN313/project_ii/project_part2/project_part2.runs/impl_1
[poise] [/home/s/s_bayaaq/COEN313/project_ii/project_part2/project_part2.runs/impl_1] > ls -al *.bit
-rw-rw---- 1 s_bayaaq s_bayaaq 3825917 Jun 28 16:29 three_k_plus_one.bit
[poise] [/home/s/s_bayaaq/COEN313/project_ii/project_part2/project_part2.runs/impl_1] >
```

Then I rename it to asm.bit as required:

```
[poise] [/home/s/s_bayaaq/COEN313/project_ii/project_part2/project_part2.runs/impl_1] > pwd
/nfs/home/s/s_bayaaq/COEN313/project_ii/project_part2/project_part2.runs/impl_1
[poise] [/home/s/s_bayaaq/COEN313/project_ii/project_part2/project_part2.runs/impl_1] > ls -al *.bit
-rw-rw---- 1 s_bayaaq s_bayaaq 3825917 Jun 28 16:38 asm.bit
-rw-rw---- 1 s_bayaaq s_bayaaq 3825917 Jun 28 16:29 three_k_plus_one.bit
[poise] [/home/s/s_bayaaq/COEN313/project_ii/project_part2/project_part2.runs/impl_1] >
```